



PhD SYNOPSIS • BOARD OF ADVANCED STUDIES & RESEARCH (BASR)

A Leakage-Safe Framework for Observability-Aware Regression Testing of Quantum Compilers

Catching the transpiler regressions the standard output check cannot observe

Furqan Nasir

PhD Scholar in Computer Science

City University of Science & IT, Peshawar

August 2026

Outline

1

Introduction

2

Literature Review

3

Proposed Methodology

4

Initial Results

5

Work Plan

6

Summary & Q&A

What This Research Is About

- A quantum program must be rewritten to run on real hardware. In Qiskit, the “transpiler” does this rewriting. [8]
- The transpiler is updated very often, and any update can accidentally introduce a regression (a bug caused by a change). [1]
- A regression can give a wrong result, a lower-quality circuit, or slower compilation.
- We need regression testing that is trustworthy and affordable — catching these before they reach users.

The Problem

1. We can't always observe the fault

- The usual check, an output-equivalence oracle, looks only at the final output. [6]
- Some regressions keep the output correct but corrupt the transpiler's internal state. [5]
- So the output-equivalence oracle cannot observe them — an observability gap.

2. We can't afford to test everything

- There are far too many tests to run them all on every update.
- Running everything each time is too slow and too expensive.
- We must select a small, high-value set — under a budget.

Seven Ways a Regression Shows Up

A single transpiler regression can surface through seven channels. The reference screen watches four of them. The other three have no standard oracle, and that blind spot is the observability gap this research targets.

WATCHED BY THE REFERENCE SCREEN

- **Semantic difference**

The compiled circuit now computes a different unitary.

- **Compilation failure**

The transpiler crashes or refuses to produce a circuit.

- **Circuit-quality regression**

The output is correct but has more gates or greater depth.

- **Performance regression**

The circuit is correct, but transpilation takes far longer.

THE OBSERVABILITY GAP

no standard oracle

- **Contract / metadata**

The output looks right, but the recorded qubit mapping is wrong.

- **Non-determinism**

Different results across runs with identical settings.

- **Global-phase discrepancy**

An operationally relevant overall phase is dropped or changed.

Research Questions

RQ1

How often do real transpiler regressions fall into fault channels not detected by a reference regression screen comprising semantic-equivalence, compilation-validity, circuit-quality, and performance oracles?

RQ2

Can real transpiler regressions that evade this reference screen be shown, from source, to be detected by fault-class-matched contract, property, isolated-pass, or determinism oracles, demonstrating that the gap is real and closable?

RQ3

Can a leakage-safe, cost-aware regression-testing pipeline for the transpiler be built and shown to be operationally sound under an executable validity-gate audit?

RQ4

(a) At feasibility scale, does the pipeline run the transparent selector and all five baselines correctly under identical budgets and anti-leakage rules? (b) In the full study, once a number of forward-regression events, fixed in advance by a prospective precision analysis, is verified, does the selector place fault-detecting tests earlier than those baselines?

Objectives

O1 → RQ1

To measure how often real transpiler regressions fall into channels the reference screen (semantic-equivalence, compilation-validity, circuit-quality, and performance oracles) does not detect, by mining merged bug-fixes and classifying each channel with inter-rater agreement (Cohen's κ for two raters, or Fleiss' κ if three or more raters participate).

O2 → RQ2

To demonstrate, from source, at least one real regression that the reference screen misses but a fault-class-matched contract, property, isolated-pass, or determinism oracle detects, establishing that the gap is real and closable.

O3 → RQ3

To build the leakage-safe, cost-aware methodology as an open prototype (cart) and show it to be operationally sound under a six-gate validity audit, verifying at least one forward regression from source.

O4 → RQ4

(a) Establish the pipeline runs the selector and all five baselines correctly under identical anti-leakage rules and budgets, and (b) commit to a powered comparison once a number of forward-regression events, fixed in advance by a prospective precision analysis, is verified, reporting the outcome either way.

Scope & Contributions

SCOPE

- **Subject:** the Qiskit 2.x transpiler — a reproducible feasibility study, not a powered evaluation.
- **Commodity only:** one ordinary laptop, circuits up to 8 qubits (H4 timed at 16–27 qubits).
- **No quantum hardware:** checking the compiler's transformation is a classical computation.
- **Deliberately bounded:** a small, honest study whose claims stay within its evidence.

CONTRIBUTIONS

- **Observability gap:** measured, and demonstrated from source (H1), as a real and common blind spot.
- **Method + prototype:** a leakage-safe, cost-aware evaluation methodology with an open harness (cart).
- **A first:** the first leakage-safe, cost-aware regression-test selection for a quantum transpiler.

The gap is reframed as: output equivalence is necessary but not sufficient for the transpiler.

What Others Have Done

- **Classical testing:** how to pick and order tests under a time budget — well studied. [3], [4]
- **Quantum testing:** finding bugs and generating tests for quantum programs. [7], [23], [24]
- **Equivalence checking:** the standard “is the answer the same?” check — efficient and trusted. [6]
- **Internal checks:** CertiQ, Giallar, and concolic testing inspect compiler passes and internal state. [26], [27], [42]

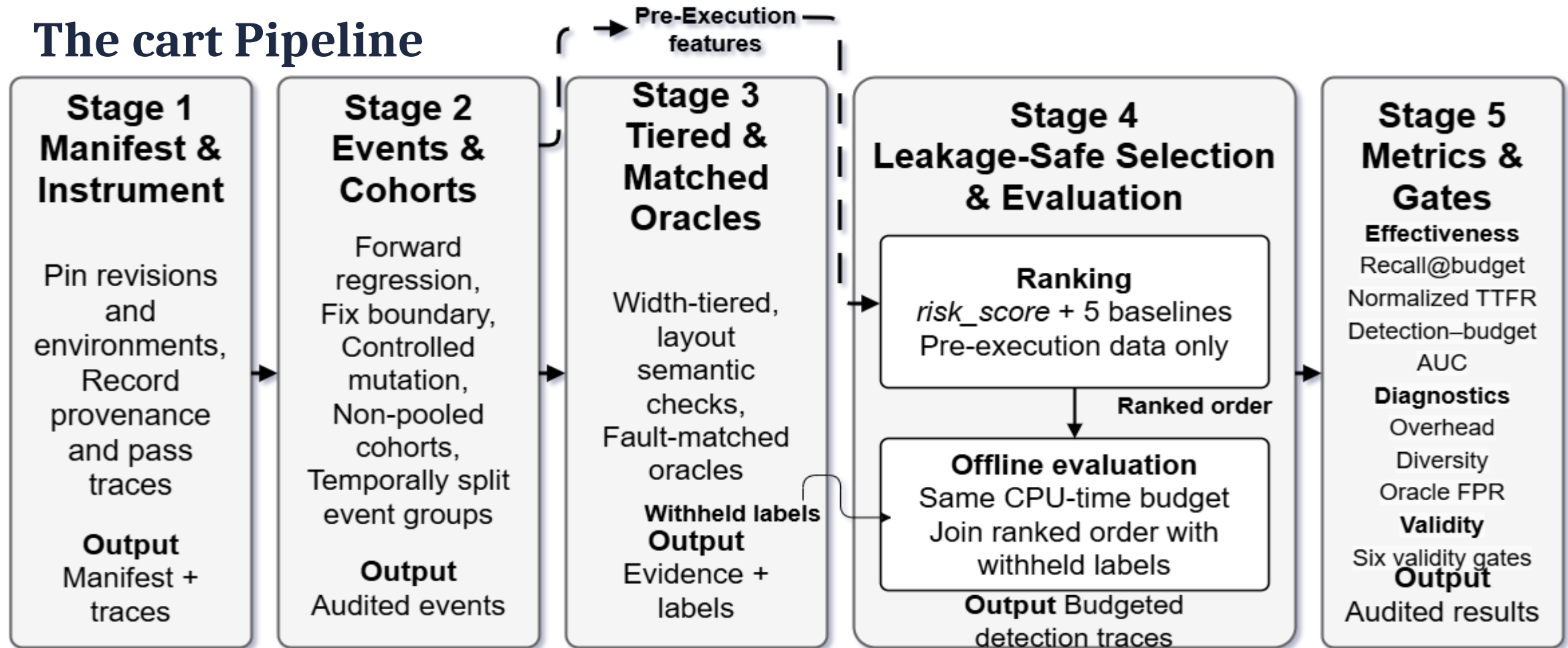
The gap we fill:

- Others verify passes and inspect internal state (Giallar, CertiQ, concolic) — so internal observability is feasible. [27], [42], [26]
- But, to our knowledge, no study turns that into matched regression oracles with budget-aware, leak-free test selection for the transpiler [3], [14].

How This Work Compares

Approach	Primary goal	QA oracle	Cost-aware	Selects / orders existing	Leakage-safe temporal	Cohort sep.
Classical RTP/TCP	prioritise tests of a program	○	●	●	◐	○
Predictive selection	learned selection at scale	○	●	●	●	○
MorphQ / MorphQ++	metamorphic bug finding	●	○	○	○	○
QuCheck	property-based testing	●	○	○	○	○
QEMI	EMI stack bug finding	●	○	○	○	○
QDiff	differential stack bug finding	●	○	○	○	○
QSPE	skeletal-program differential testing	●	○	○	○	○
Giallar	formal Qiskit-pass verification	●	○	○	○	○
Benchpress	SDK compilation benchmarking	◐	○	○	○	○
This work	cost-aware test selection for the transpiler	●	●	●	●	●

The cart Pipeline



Shared Reproducibility and Validity Substrate

Two-layer environment • write-once artifacts • CPU calibration • frozen stage map • temporal split • claim scope

Which Oracle Detects Which Fault?

No single oracle covers all channels — each fault channel needs its matched oracle. “Output oracle” is not one thing.

Manifestation	Unitary equiv.	Compilation	Quality	Performance	Contract	Determinism
Semantic difference	Yes	Sometimes	No	No	No	No
Compilation failure	N/A	Yes	No	No	No	No
Circuit-quality regression	No	No	Yes	No	No	No
Performance regression	No	No	No	Yes	No	No
Contract/metadata corruption	No	No	No	No	Yes	Sometimes
Non-determinism	Not in 1 run	No	Sometimes	Sometimes	Sometimes	Yes

Contract and determinism channels are not covered by the reference black-box regression screen. The framework operationalises matched checks for them — and we demonstrate that a matched contract oracle can close the gap for H1.

The Test Selector (risk_score)

Rank every candidate test by its expected value per unit cost, then run the highest-scoring first.

VALUE how much a test is worth running

chance it catches the fault × how much the fault matters ×
how trustworthy the check is × novelty

factors: change-stage match · circuit/backend sensitivity · criticality · historical risk · oracle confidence · novelty



COST

expected time to run

transpilation + oracle time, measured on the baseline

The higher a test's value per unit cost, the earlier it runs — under a fixed test budget.

- **Transparent:** hand-set weights, no black-box AI — every choice is explainable.
- **Leakage-safe:** uses only pre-execution information, and never peeks at the outcome.
- **Frozen & compared:** weights fixed before evaluation, tested against five baselines including random, and checked by ablation.
- **Stress-tested:** conclusions hold under ±50% weight perturbation and a 10–20% reserve sweep, so no single hand-set number drives the result.

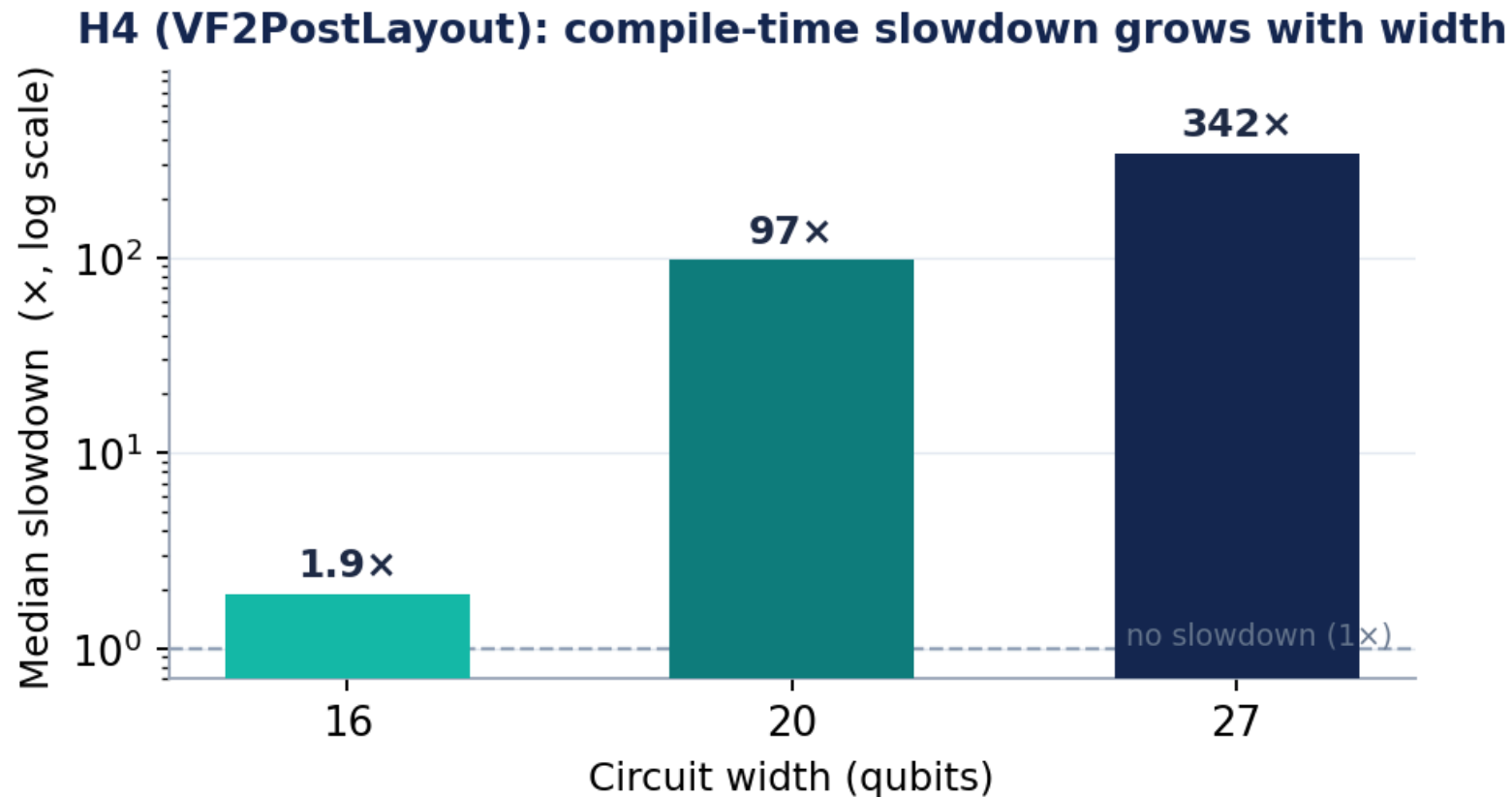
What We Tested

- Run on a single ordinary laptop, using circuits of up to eight qubits.
- We ran the whole regression-testing pipeline end to end.
- We also reproduced real historical transpiler regressions from Qiskit's own history.

Historical Transpiler Regressions Attempted

Event (PR)	Cohort	Fault type ->channel	Status
H4 (#14120)	forward_regression	Performance	verified (multi-run protocol)
H1 (#14603)	fix_boundary	semantic ->contract/metadata	demonstrated gap (output-unobservable)
H2 (#14919)	fix_boundary	semantic -> contract/metadata	not run
H3 (#14730)	fix_boundary	quality -> determinism	blocked (needs noisy target)
H5 (#16285)	fix_boundary	functional -> compilation_failure	not run (input-specific crash)

H4 — A Verified Performance Regression



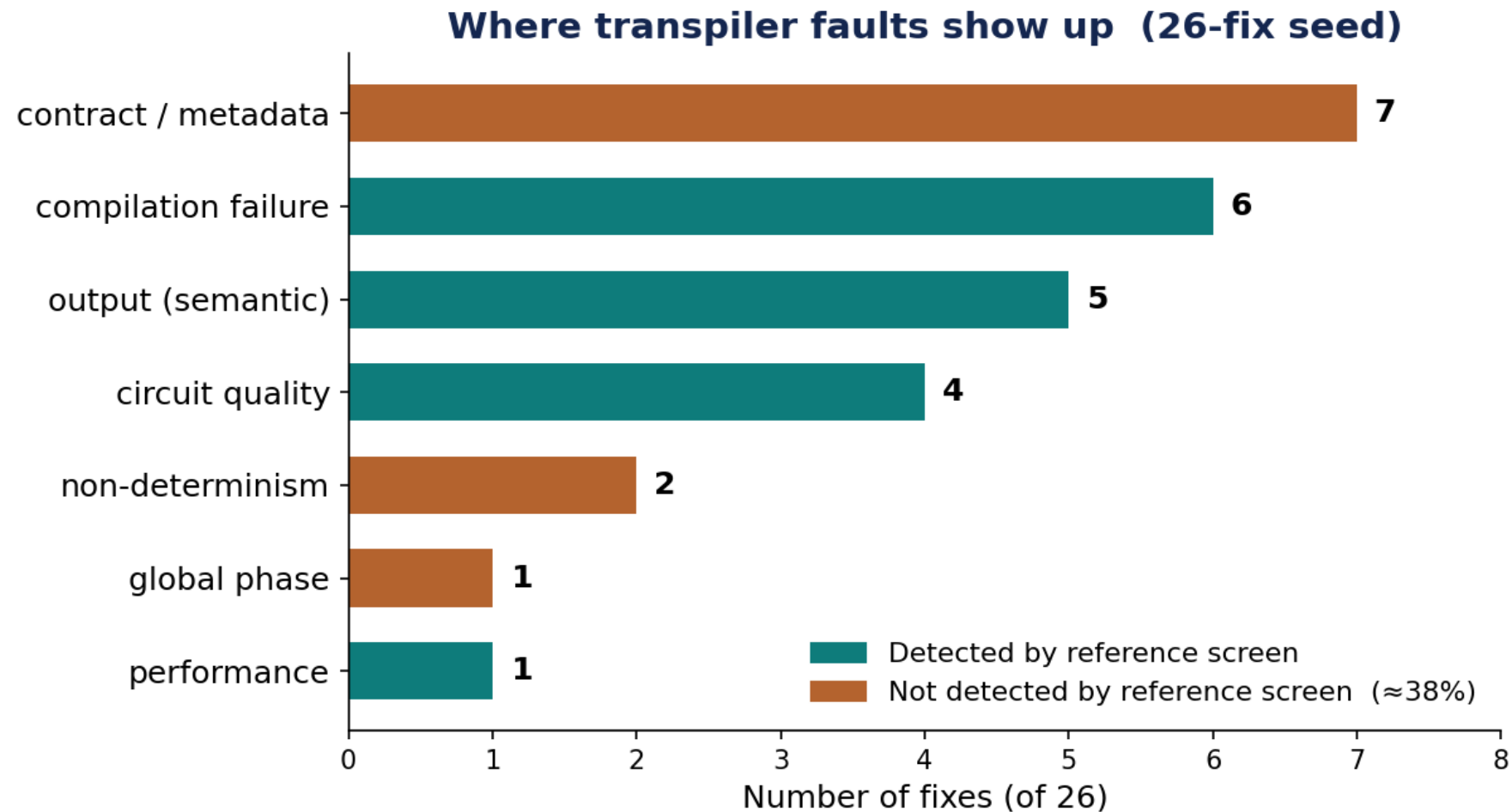
H4's width-extended run (16–27 qubits) is a separate regime from the primary corpus (≤ 8 qubits).

H1 — The Observability Gap on a Real Regression

Oracle	Result on the buggy build
Black-box output-equivalence	EQUIVALENT (fault missed)
Property / isolated-pass (matched)	contract_metadata_fail (fault detected)

INITIAL RESULTS

Preliminary Oracle-Observability Gap in the 26-Fix Seed



Work Plan

Phase 1	Feasibility study (methodology, prototype, initial results)	Completed
Phase 2	Scale-up: verify ≥ 3 forward regressions, expand mining	2026 Q3
Phase 3	Comparison: selector vs baselines	2026 Q4
Phase 4	Validation and cross-SDK generalisation	2027 Q1
Phase 5	Thesis write-up and defence	2027 Q2

SUMMARY

Where We Stand

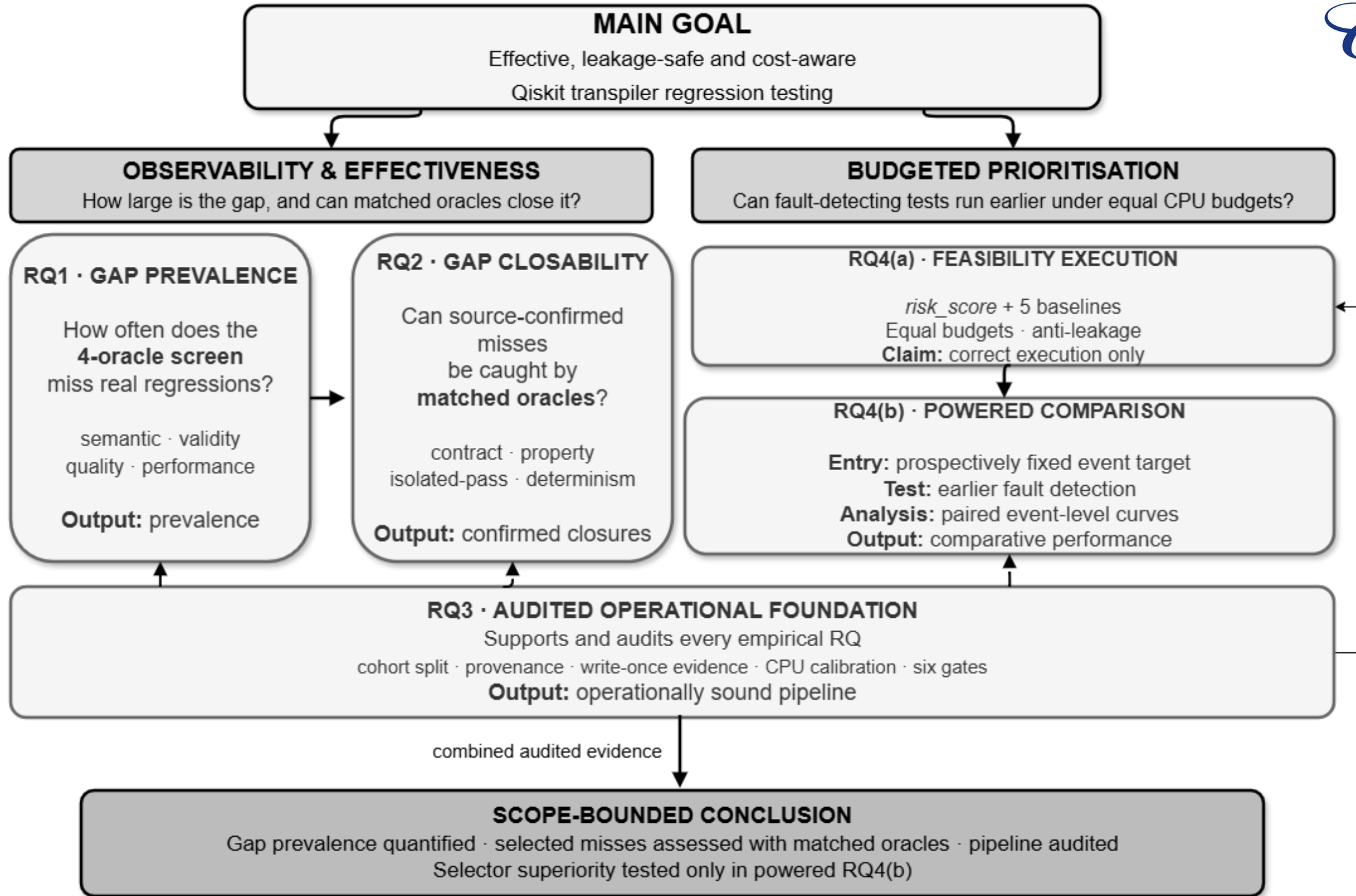
- The prototype is feasible and has produced usable evidence on two real historical regression events.
- One real contract regression (H1) shows the reference screen can miss a fault a matched oracle detects.
- The 26-fix seed indicates a potentially material gap. Full mining and the comparative selector evaluation remain future work.

Thank you — Questions & discussion

REFERENCES

Selected References

- [1] Paltenghi & Pradel. Bugs in quantum computing platforms: an empirical study. OOPSLA, 2022. arXiv:2110.14560
- [3] Yoo & Harman. Regression testing minimization, selection and prioritisation: a survey. STVR, 2012. doi:10.1002/stvr.430
- [4] Machalica et al. Predictive test selection. ICSE-SEIP, 2019. arXiv:1810.05286
- [5] Paltenghi & Pradel. A survey on testing and analysis of quantum software. arXiv:2410.00650, 2024.
- [6] Burgholzer & Wille. Advanced equivalence checking for quantum circuits. IEEE TCAD, 2021. doi:10.1109/TCAD.2020.3032630
- [7] Paltenghi & Pradel. MorphQ: metamorphic testing of the Qiskit platform. ICSE, 2023. doi:10.1109/ICSE48619.2023.00202
- [8] Javadi-Abhari et al. Quantum computing with Qiskit. arXiv:2405.08810, 2024.
- [14] Elsner et al. Readily available information for regression test optimization in CI. ISSTA, 2021. doi:10.1145/3460319.3464834
- [22] Pontolillo et al. QuCheck: property-based testing for Qiskit. arXiv:2503.22641, 2025.
- [23] Luo et al. QEML: testing quantum stacks via equivalence modulo inputs. FASE, 2026. arXiv:2602.09942
- [24] Wang et al. QDiff: differential testing of quantum software stacks. ASE, 2021. doi:10.1109/ASE51524.2021.9678792
- [25] Ye et al. QSPE: enumerating skeletal quantum programs. arXiv:2602.00024, 2026.
- [26] Choudhury et al. Concolic testing for quantum compilers. IEEE ICCD, 2025. doi:10.1109/ICCD65941.2025.00057
- [27] Tao et al. Giallar: push-button verification for the Qiskit compiler. PLDI, 2022. doi:10.1145/3519939.3523431
- [42] Shi et al. Contract-based verification of a realistic quantum compiler (CertiQ). arXiv:1908.08963, 2019.
- [46] Oldfield et al. Faster & better quantum software testing. ACM TOSEM, 2025. doi:10.1145/3714468
- [47] Kang, Lee & Oh. Statistical testing of quantum programs. OOPSLA, 2024. doi:10.1145/3689716
- [48] Lee, Hong & Ryu. Exact inference for quantum circuits. ASE, 2025. doi:10.1109/ASE63991.2025.00203
- [50] Burgholzer, Raymond & Wille. Verifying results of the IBM Qiskit compilation flow. QCE, 2020. doi:10.1109/QCE49297.2020.00051
- [51] Ali et al. Input/output coverage criteria for testing quantum programs. ICST, 2021. doi:10.1109/ICST49551.2021.00014
- [52] Wang et al. QuSBT: search-based testing of quantum programs. ICSE-C, 2022. doi:10.1145/3510454.3516839
- [53] Klimis et al. Shaking up quantum simulators with fuzzing and rigour. OOPSLA, 2025. doi:10.1145/3763100
- [54] Mei et al. Equivalence checking via path-sum & weighted model counting. TACAS, 2026. doi:10.1007/978-3-032-22749-2_21
- [55] Ye et al. Is measurement enough? Rethinking output validation in quantum program testing. ASE (NIER), 2025. arXiv:2509.16595



Pipeline Feasibility: Validity Gates

Implementation-validity gate	Result
Budget compliance	PASS
Reproducibility (same seed $\Rightarrow$ same ranking)	PASS
Leakage absence (current-event outcome cannot change ranking)	PASS
Label reproducibility from write-once artifacts	PASS
Metric correctness vs hand-computed cases	PASS
Oracle coverage	PASS

risk_score — the Formal Definition

$$\text{risk_score} = (\varepsilon + \text{CSM}) \times \text{CBS} \times \text{IP} \times \text{HR} \times \text{OC} \times \text{NM} \div \text{EC}$$

A transparent, hand-set heuristic — an interpretable score, not a calibrated probability.

ε = small additive smoothing constant

CSM = change-stage match (1 if the modified stage is exercised, else 0)

CBS = circuit-backend sensitivity

IP = impact prior (circuit criticality)

HR = historical risk (1 + prior-failure-rate)

OC = oracle confidence

NM = novelty multiplier (down-weights repeats)

EC = expected cost (transpilation + oracle time)

Classical Foundation vs. Quantum-Specific Need

Dimension	Classical testing	Quantum transpiler
Observed result	Deterministic, directly inspectable	Probabilistic samples, internal state not directly observable
Correctness	Equality / observational equivalence	Up to global phase & qubit/layout permutation
Fault model	Crashes, wrong values, performance	+ phase, entanglement, mapping/routing, backend/noise
Execution cost	Repeatable CPU time, one run	Repeated shots, simulation growth, oracle cost
Regression oracle	Expected output, assertions, logs	Output equivalence alone misses metadata/layout/quality
Foundation	Selection, prioritisation, cost, APFD/APFDc	Retained — but features, costs, oracles redefined

Classical principles are the foundation, but they do not transfer unchanged — this work is a quantum-specific adaptation and empirical validation, not the invention of regression testing.

Evidence: Why Quantum Differs

- **39.9% of real quantum-platform bugs are quantum-specific** — concentrated in compile/optimize components — a classical fault model is incomplete [1].
- **Quantum outputs are exponentially large distributions** — conventional distribution testing can miss phase-flip faults [46].
- **State equivalence can't be read from one observation** — assurance needs repeated measurement, and specialised oracles expose non-crash bugs [47], [48].
- **Equivalence checking had to be redesigned** — for reversibility, superposition, entanglement — not reused from classical [6].
- **Compiler-pass reasoning must encode quantum semantics** — formal verification confirms this is necessary and feasible [27].

Central claim: classical machinery does not directly solve quantum-transpiler regression testing — its observability, oracle, equivalence and cost models do not hold unchanged.

How Do We Evaluate Something Novel?

The method is new — the evaluation apparatus is entirely standard. We do not compare against a non-existent rival, so we do what every regression-testing study does:

- **Established baselines:** the selector is compared against random ordering plus cost-, history-, coverage/diversity- and change-based heuristics — the field-standard set [3], [10], [14].
- **Standard metrics:** APFD and cost-cognizant APFDc, recall@budget, detection-vs-budget AUC, oracle false-positive rate, inter-rater κ — all from the literature [10], [12], [40], [41].
- **Real ground truth:** evaluation on real, historical Qiskit regressions where the correct answer is known — e.g., a bug the output oracle passed but the matched oracle caught.
- **Ablation:** quantum-specific components are switched off one at a time to show which actually carry the effect — the method is compared against reduced versions of itself.
- **Self-audit:** six leakage-safe validity gates make every result reproducible and trustworthy, independent of any competitor.
- **Unit of analysis:** the regression event is the independent unit, and test-oracle pairs are nested observations, not independent replications.

Where the Tested Regressions Come From

The events (e.g., H1 = PR #14603, H4 = PR #14120) are not “just GitHub” — for an open-source SDK the repository IS the authoritative primary record.

- **Canonical primary source:** merged pull requests, their linked issues, and release notes are the official record of what changed and why — permanent and citable.
- **Standard methodology:** mining merged bug-fix commits is established empirical-software-engineering practice [49] (the SZZ / Mining-Software-Repositories line).
- **Same source as accepted benchmarks:** Bugs4Q curates real Qiskit bugs from this history [29], and Paltenghi & Pradel mined 223 real platform bugs the same way [1].
- **Full traceability:** each event → inducing/fixing PR + issue thread + release note. All IDs, revisions and build scripts released for reproduction.
- **Why not RFCs:** Qiskit RFCs cover major design changes, not routine bug fixes — those live in the PR + release-note record, which is the correct provenance.

Every tested regression is traceable to a permanent, verifiable record — more so than most secondary sources.

Do We Run Quantum Simulations? (No Hardware)

Yes — classical simulation only, on the CPU. A simulator is classical software that computes what a quantum device would do, and it never needs a quantum device.

- **What we test is classical:** the transpiler is compiler software, so checking its output is a classical computation, not a physics experiment.
- **How the check runs:** for small circuits (≤ 12 qubits) we compute the exact unitary with linear algebra (Qiskit's classical simulator) and compare up to global phase [6].
- **It runs on a laptop:** the corpus uses up to 8 qubits, so every semantic check simulates in milliseconds on a 32 GB CPU — no device, no queue, no noise.
- **Where simulation would blow up:** above the cap we do NOT simulate — structural checks (basis + coupling-map) take over, and even the 27-qubit performance test only times the compiler, not the circuit's output.
- **No hardware on the critical path:** our research validates the compiler transformation, not physical execution — it stops one step before hardware is needed. Physical-noise fidelity is a separate, downstream question, out of scope.

Simulator / hardware: it runs on an ordinary CPU. Exact simulation is exponential, so the design caps it and falls back — that is why no quantum device is needed.

Precedent: Testing Quantum Compilers Without Hardware

Top-venue studies evaluate quantum compilers and quantum-software testing in classical software or simulation — none required a QPU.

Study	Venue (year)	Tier	Ran on	QPU?
Giallar [27]	PLDI 2022	A*	symbolic pass verification (classical)	No
VOQC [44]	POPL 2021	A*	Coq proofs over circuit semantics	No
Burgholzer et al. [50]	IEEE QCE 2020	Nat.	equivalence checking of Qiskit compilation	No
MorphQ [7]	ICSE 2023	A*	Qiskit simulator (8,000+ program pairs)	No
Muskit [30]	ASE 2021	A*	Qiskit software (mutation analysis)	No
Bugs4Q [29]	ASE 2021	A*	Qiskit buggy/fixed revisions	No
QMutPy [31]	IEEE TQE 2022	Q1	Qiskit software (24 programs)	No
Coverage criteria [51]	ICST 2021	A	simulator + statistical oracles	No
QuSBT [52]	ICSE-C 2022	A*	Qiskit as the simulation framework	No
Simulator fuzzing [53]	OOPSLA 2025	A*	Qiskit/Cirq simulators (800,000+ circuits)	No
Exact inference [48]	ASE 2025	A*	classical exact inference oracle	No

Tier — conferences: CORE 2023 (A = top tier, A = strong). Journals: Scimago quartile (Q1 = top 25%). IEEE QCE is a specialised quantum venue (CORE national tier).*

Our study is in exactly this established, hardware-free mode — evaluating the compiler transformation, not physical execution.

Why Qiskit? A Fast-Moving Transpiler

Qiskit ships a new minor version about every three months — and almost every release changes the transpiler.

RELEASE VELOCITY (2.x series)

- **~3-month cadence:** a new minor version on a roughly quarterly schedule (official policy).
- **2.0 → 2.4:** five minor versions across ~2025 to mid-2026.
- **Plus patches:** 2.4.0 → 2.4.1 → 2.4.2 in quick succession.

NEARLY EVERY RELEASE TOUCHES THE TRANSPILER

- **v2.2:** transpilation 10–20% faster, standalone transpiler in C.
- **v2.3:** VF2Layout and VF2PostLayout reworked, custom transpiler passes in C.
- **v2.4:** faster fault-tolerant transpilation pipeline, DAG-level transpiler passes.

The passes we study are live targets: VF2PostLayout — the site of our H4 regression — was itself reworked in v2.3.

A high-churn transpiler makes every release a fresh regression opportunity, which is exactly why Qiskit — the most actively developed quantum compiler, with the richest public record of real fixes to mine — is the right subject.